

Ejercicio 6: Análisis de Cueva (Divide y Vencerás)

Enunciado

Te dan una imagen de una cueva que tiene n píxeles de altura (filas) y m píxeles de anchura (columnas). Cada píxel es un 0 o un 1. Cada columna de la imagen pertenece exactamente a una de las siguientes categorías:

- **Columna mineral:** Cada píxel en ella es un 1.
- **Estalactita:** Tiene 0 y 1, y todos los unos aparecen encima de todos los 0.
- **Estalagmita:** Tiene 0 y 1, y todos los unos aparecen debajo de todos los 0.
- **Nada:** Cada píxel en ella es un 0.

La longitud de una estalactita o estalagmita es el número de 1 en esa columna. Dada tal imagen, diseña un algoritmo de coste en tiempo en $\mathcal{O}(m \log n)$ que genere el siguiente valor:

$\max(\text{longitud de la estalactita más larga, longitud de la estalagmita más la}$

1. Estrategia de Divide y Vencerás

El problema nos pide procesar una matriz de tamaño $n \times m$. Dado que la complejidad objetivo es $\mathcal{O}(m \log n)$, disponemos de un tiempo de $\mathcal{O}(\log n)$ **para analizar cada columna**.

Identificación de la Categoría en $\mathcal{O}(1)$

Podemos determinar con total certeza a qué categoría pertenece cualquier columna j examinando únicamente su primer píxel $\text{imagen}[0][j]$ y su último píxel $\text{imagen}[n-1][j]$:

1. Si $\text{imagen}[0][j] == 1$ y $\text{imagen}[n-1][j] == 1 \implies$ **Columna mineral** (todos son 1, longitud = n).
2. Si $\text{imagen}[0][j] == 0$ y $\text{imagen}[n-1][j] == 0 \implies$ **Nada** (todos son 0, longitud = 0).
3. Si $\text{imagen}[0][j] == 1$ y $\text{imagen}[n-1][j] == 0 \implies$ **Estalactita** (los 1 están arriba, los 0 abajo).
4. Si $\text{imagen}[0][j] == 0$ y $\text{imagen}[n-1][j] == 1 \implies$ **Estalagmita** (los 0 están arriba, los 1 abajo).

Búsqueda de la Longitud en $\mathcal{O}(\log n)$

Una vez identificada la categoría, podemos encontrar el punto de transición exacto entre 1 y 0 aplicando **Búsqueda Binaria** (Divide y Vencerás) sobre las filas de la columna:

- **Para una Estalactita:** Buscamos el **primer** 0 de la columna. El índice de este primer 0 (0-indexado) coincidirá exactamente con la cantidad de 1s (longitud de la estalactita).

- **Para una Estalagmita:** Buscamos el **primer 1** de la columna. La longitud de la estalagmita será $n - \text{índice del primer 1}$.

2. Algoritmo y Pseudocódigo

Búsqueda Binaria para Estalactita (Buscar primer 0)

Busca de forma binaria la transición de 1 a 0 en la columna *col* entre las filas *inicio* y *fin*:

```

Función BuscarPrimerCero(imagen, col, inicio, fin):
    Si inicio > fin:
        Retornar inicio // Punto de transición

    medio = (inicio + fin) / 2

    Si imagen[medio][col] == 0:
        // El primer cero está en 'medio' o a su izquierda
        Retornar BuscarPrimerCero(imagen, col, inicio, medio - 1)
    Sino:
        // 'medio' sigue siendo un 1, el primer cero está a la derecha
        Retornar BuscarPrimerCero(imagen, col, medio + 1, fin)

```

Búsqueda Binaria para Estalagmita (Buscar primer 1)

Busca de forma binaria la transición de 0 a 1 en la columna *col* entre las filas *inicio* y *fin*:

```

Función BuscarPrimerUno(imagen, col, inicio, fin):
    Si inicio > fin:
        Retornar inicio // Punto de transición

    medio = (inicio + fin) / 2

    Si imagen[medio][col] == 1:
        // El primer uno está en 'medio' o a su izquierda
        Retornar BuscarPrimerUno(imagen, col, inicio, medio - 1)
    Sino:
        // 'medio' sigue siendo un 0, el primer uno está a la derecha
        Retornar BuscarPrimerUno(imagen, col, medio + 1, fin)

```

Algoritmo Principal

```

Función CalcularMaximoElemento(imagen, n, m):
    max_estalactita = 0
    max_estalagmita = 0

    Para j desde 0 hasta m - 1:
        primer_pixel = imagen[0][j]
        ultimo_pixel = imagen[n-1][j]

        // 1. Identificar categoría y resolver en O(log n)
        Si (primer_pixel == 1 y ultimo_pixel == 0):

```

```

// Caso Estalactita
longitud = BuscarPrimerCero(imagen, j, 0, n - 1)
max_estalactita = Maximo(max_estalactita, longitud)

Sino si (primer_pixel == 0 y ultimo_pixel == 1):
// Caso Estalagmita
primer_uno = BuscarPrimerUno(imagen, j, 0, n - 1)
longitud = n - primer_uno
max_estalagmita = Maximo(max_estalagmita, longitud)

Retornar Maximo(max_estalactita, max_estalagmita)

```

3. Análisis de Complejidad

- **Complejidad Temporal:**
 - **Análisis de una columna:** Determinar la categoría toma $\mathcal{O}(1)$ tiempo. La búsqueda binaria sobre una columna de altura n divide el espacio de búsqueda a la mitad en cada paso, costando exactamente $\mathcal{O}(\log n)$ comparaciones.
 - **Coste total:** Al repetir este proceso para las m columnas, el coste temporal total es estrictamente de $\mathcal{O}(m \log n)$, cumpliendo con la restricción del enunciado.
- **Complejidad Espacial:**
 - $\mathcal{O}(1)$ (espacio auxiliar constante) si implementamos las búsquedas binarias de forma iterativa, o bien $\mathcal{O}(\log n)$ de espacio en la pila de llamadas del sistema si usamos las funciones recursivas propuestas.